



Introduction to SIMD



Demo

Overview

- What is SIMD?
- How does SIMD work on a hardware level?
- How does SIMD make programs run faster?
- How to get started writing SIMD code?

Single instruction, multiple data (SIMD)

A way to utilize the CPU hardware to process multiple values simultaneously for better performance.

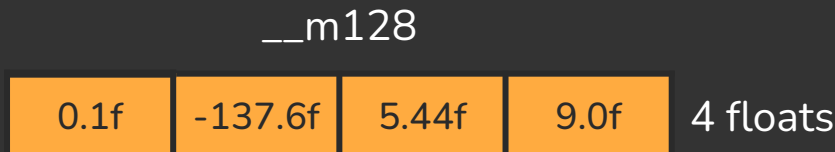


Vector/vectorization in the context of SIMD

Think of a SIMD vector as a fixed-size array.

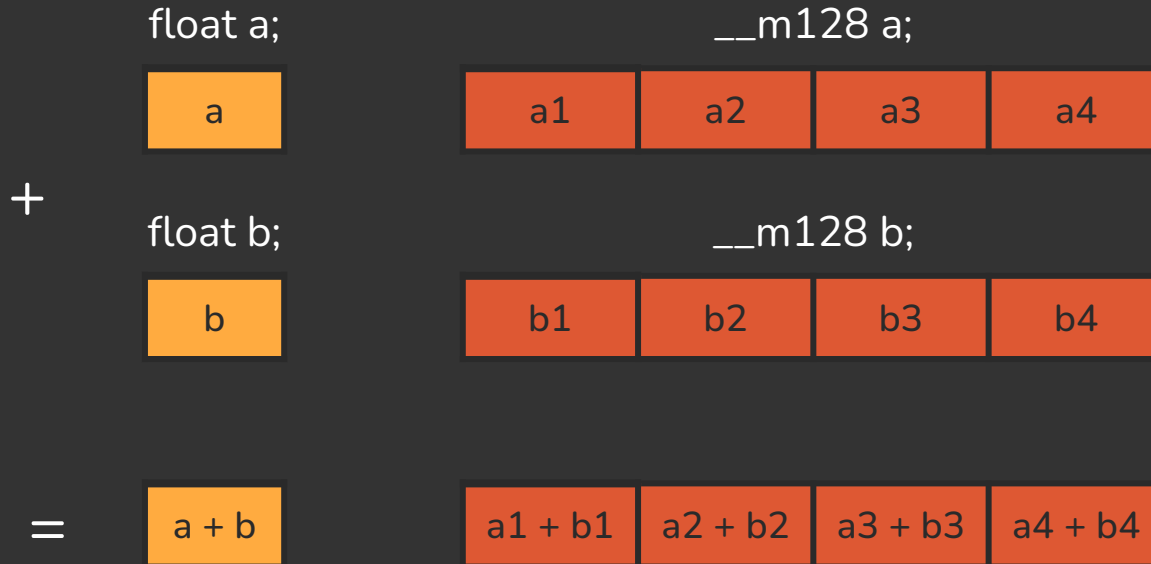
No relation to:

- `std::vector`
- Mathematical vector
- `Vector2`, `Vector3`



Each slot is called a "lane"

Scalar vs. Vector



Areas of usage

Performance sensitive applications that process data in batches in a way that can be parallelized.

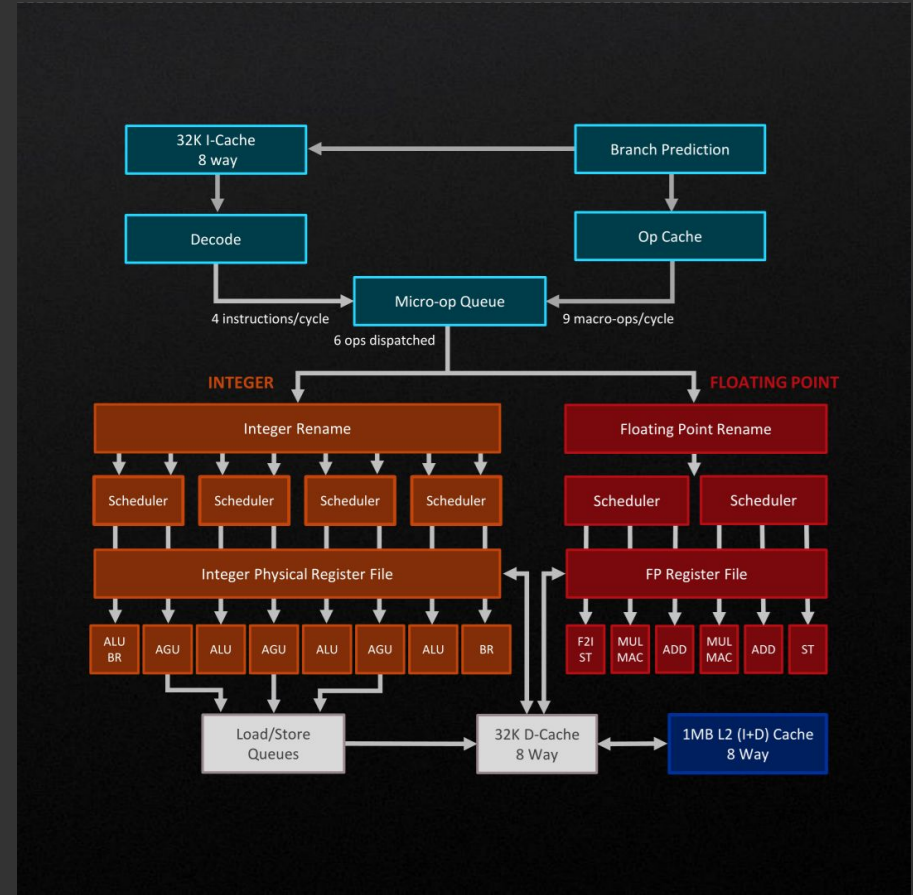
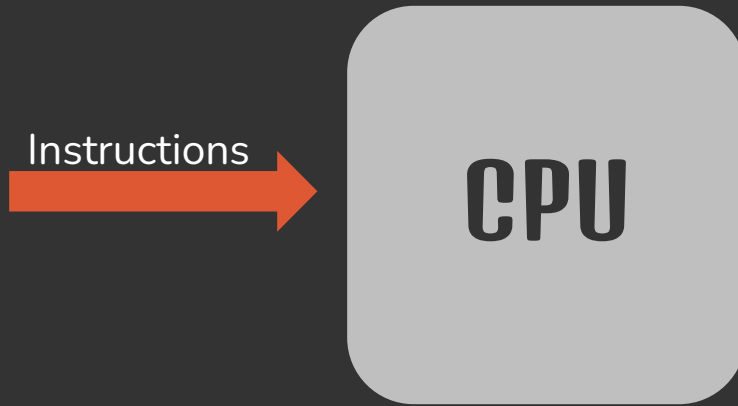
- Video codecs
- Photoshop
- Cryptography
- Game engines

simdjson speed (C++)

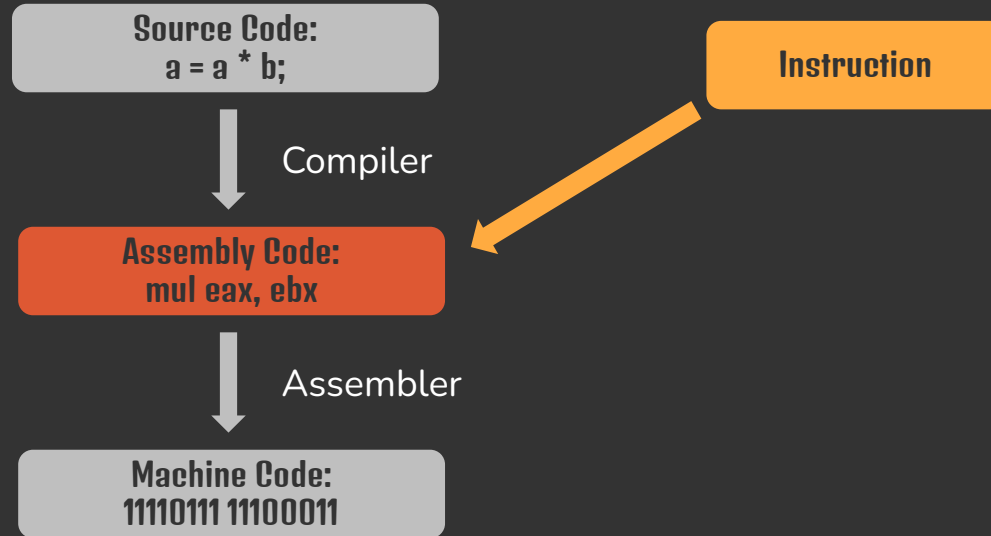
twitter.json: 2.4 GB/s on 3.4 GHz Skylake

	speed
simdjson (C++)	2.4 GB/s
RapidJSON (C++)	0.65 GB/s
Jackson (Java)	0.35 GB/s
readLines C++	1.5 GB/s
disk	2.2 GB/s

CPU Architecture Overview

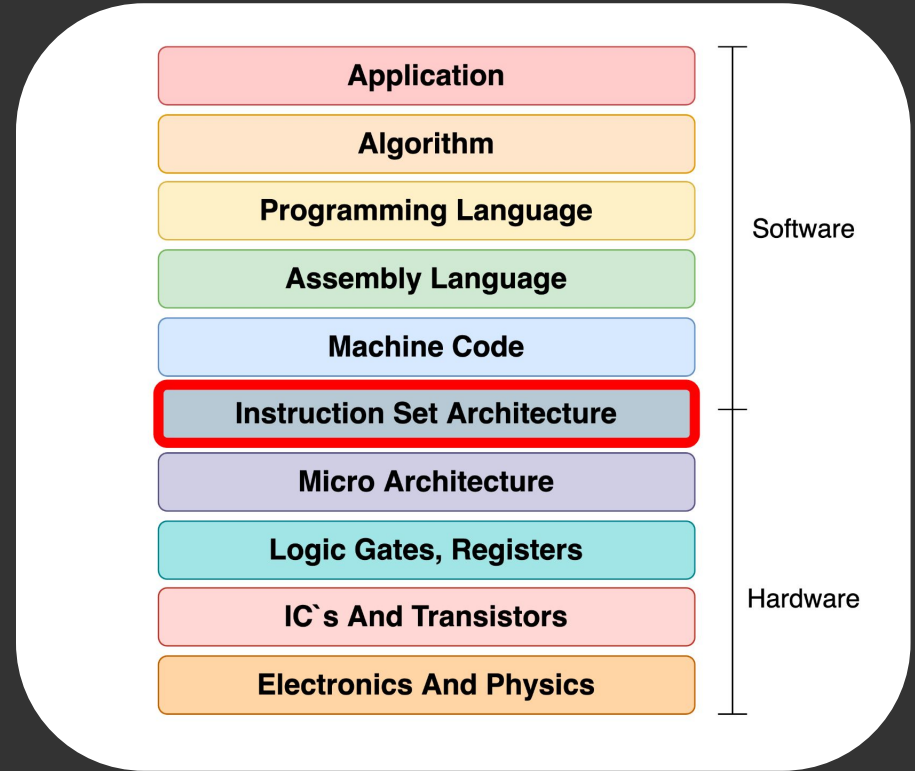


What is an instruction?



Instruction Set Architecture (ISA)

- x86 - Desktop, PS5, Xbox Series X
 - Intel
 - AMD
- ARM - Mobile, Switch
 - Apple Silicon
 - Qualcomm Snapdragon



Instruction set extensions

SIMD instructions aren't generally part of the base instruction set. Instead, they come in the form of extensions.

- x86
 - SSE
 - AVX
- ARM
 - NEON

Anatomy of an x86 instruction

add eax, ebx



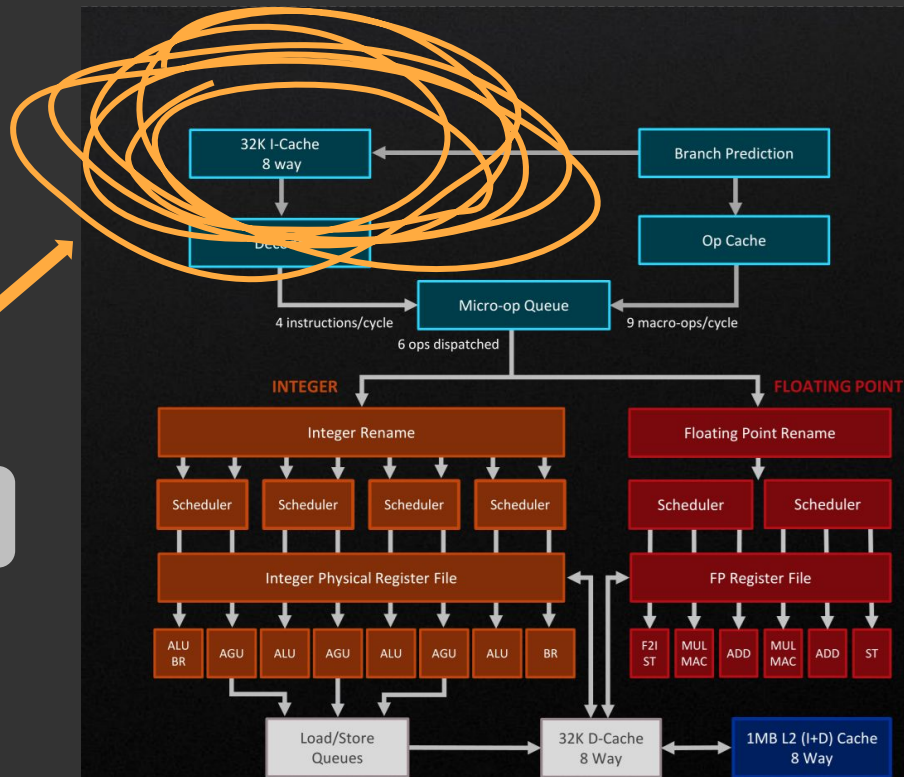
Instruction cycle



Fetch

Retrieve the next instruction(s) from memory.

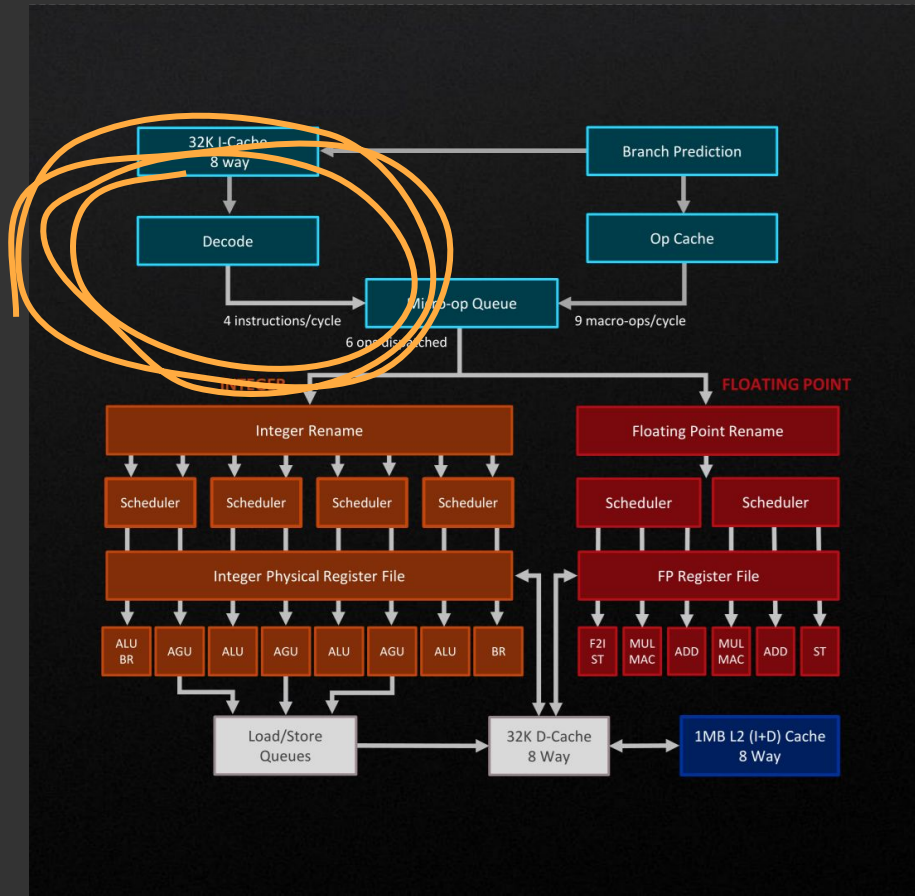
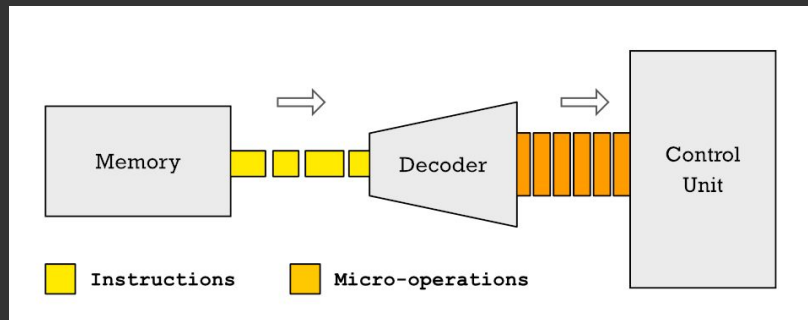
Instruction Cache



Decode

Interpret the instruction, breaking it down into specific operations.

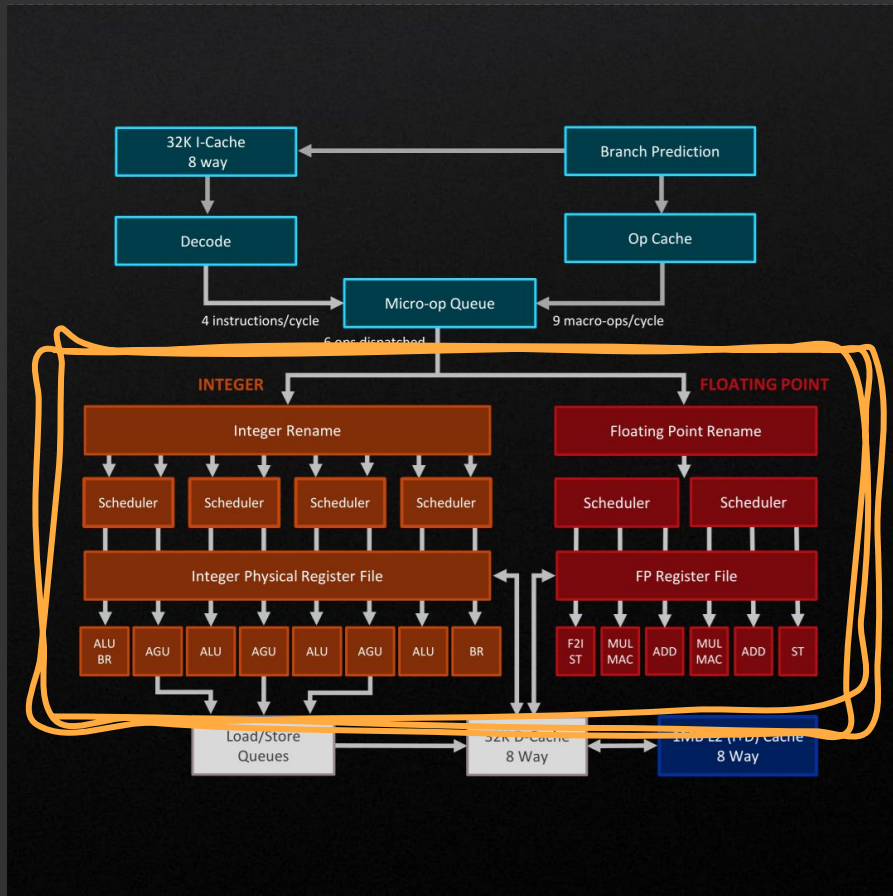
Decoders



Execute

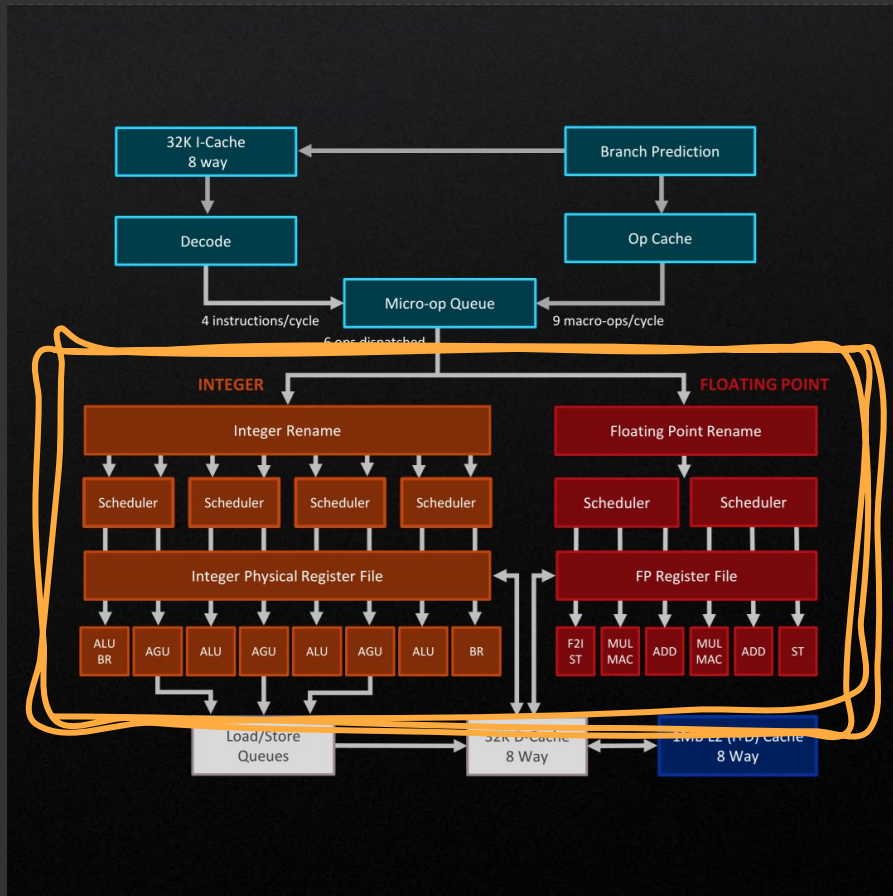
Carry out the operations specified by the instruction.

Execution Engine



Execute

- Registers - circuits that temporarily store inputs and outputs
- Execution units - circuits that perform operations on data in registers

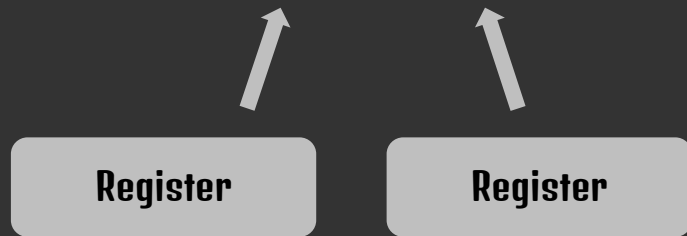


Registers

A register is a super fast storage location inside the CPU. It's like a workspace for the CPU that temporarily hold data for processing.



add eax, ebx



Execution units

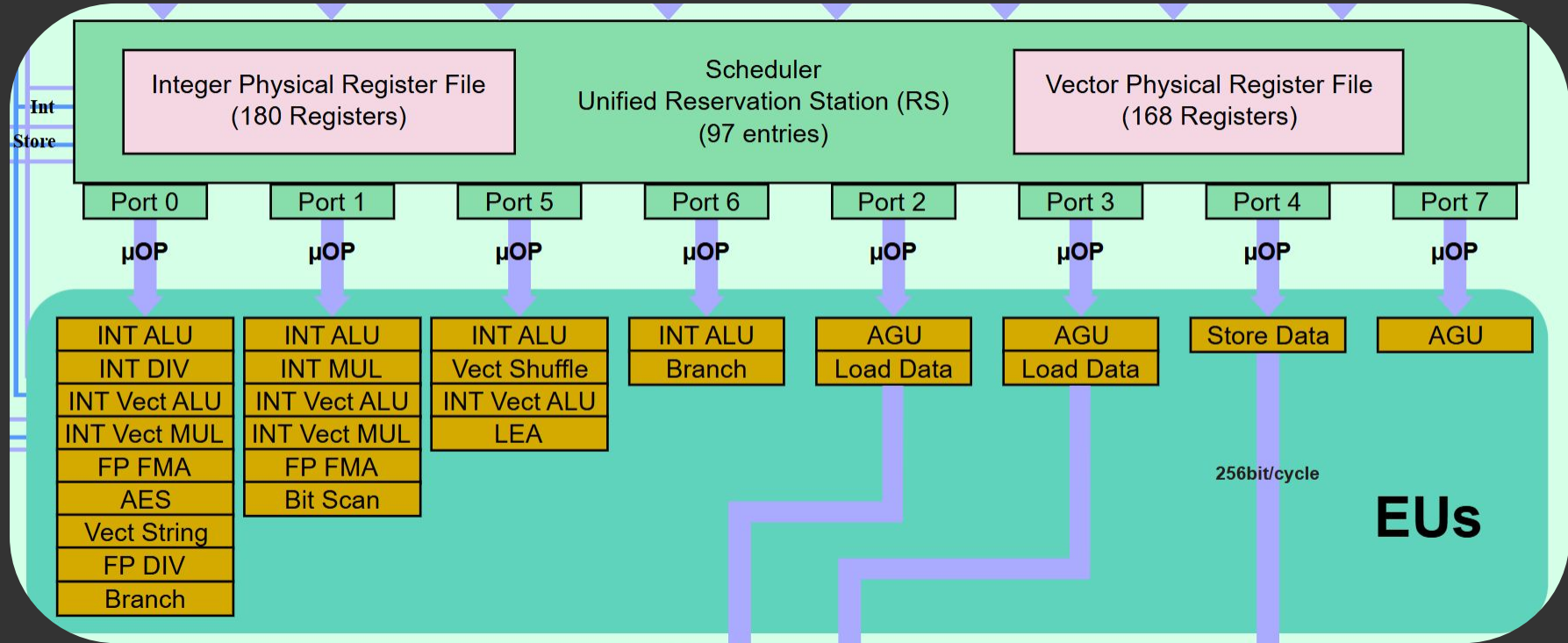
Specialized components for executing different types of operations.

- Arithmetic logic unit (ALU)
- Address generation unit (AGU)
- Load-store unit (LSU)
- Branch execution unit (BEU)



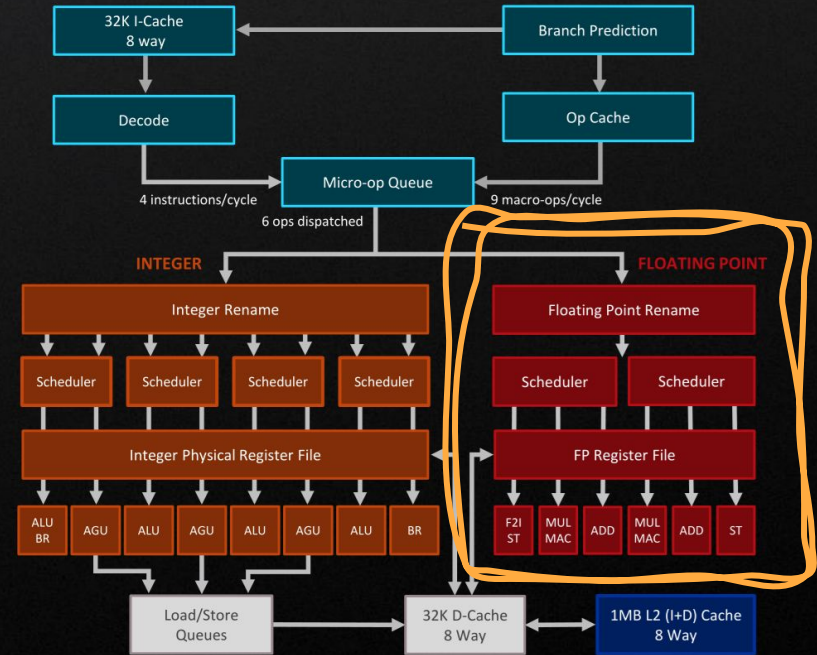
Where is the SIMD hardware?

It's been here all along!

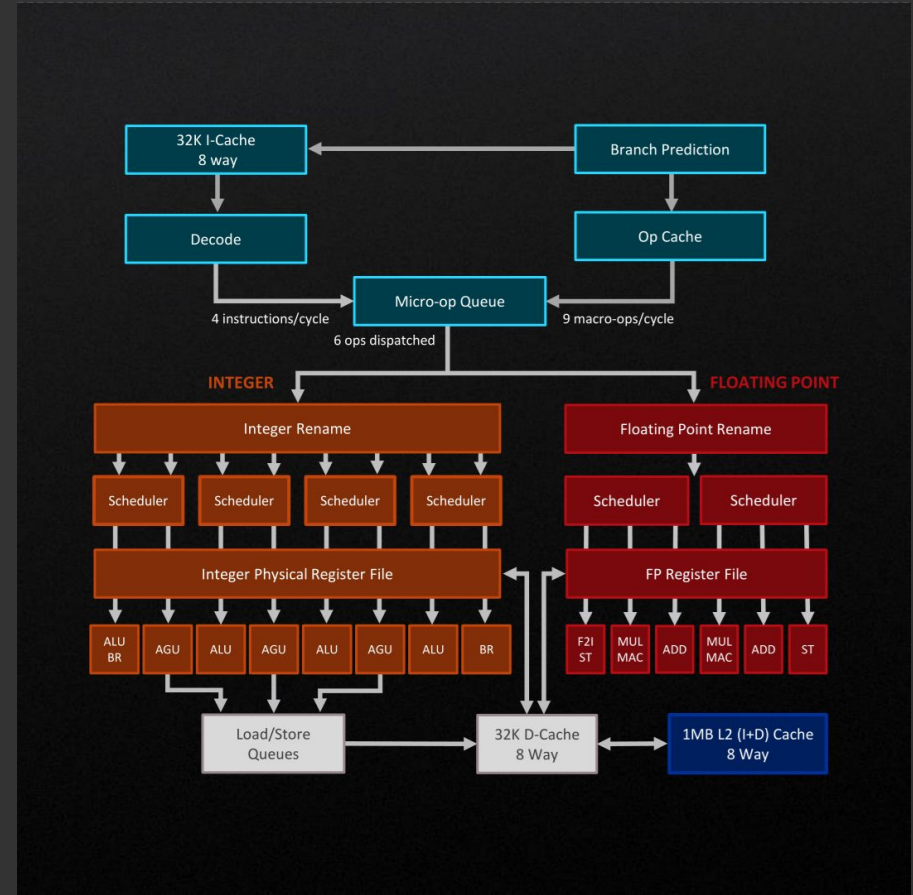
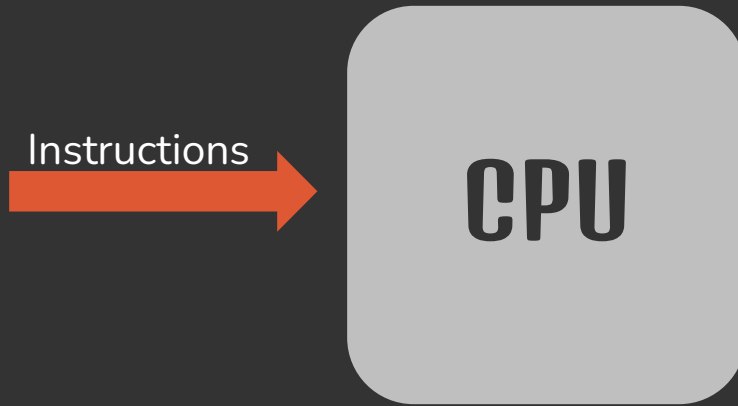


FP is always SIMD (sort of)

Modern compilers will generate SSE instructions for scalar floating point operations.

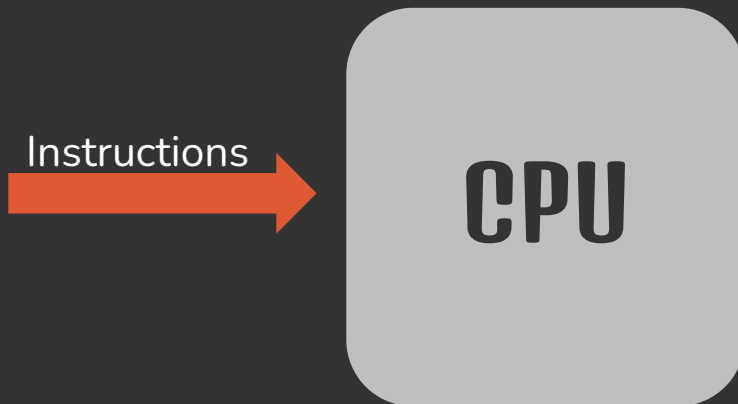


CPU Architecture Overview



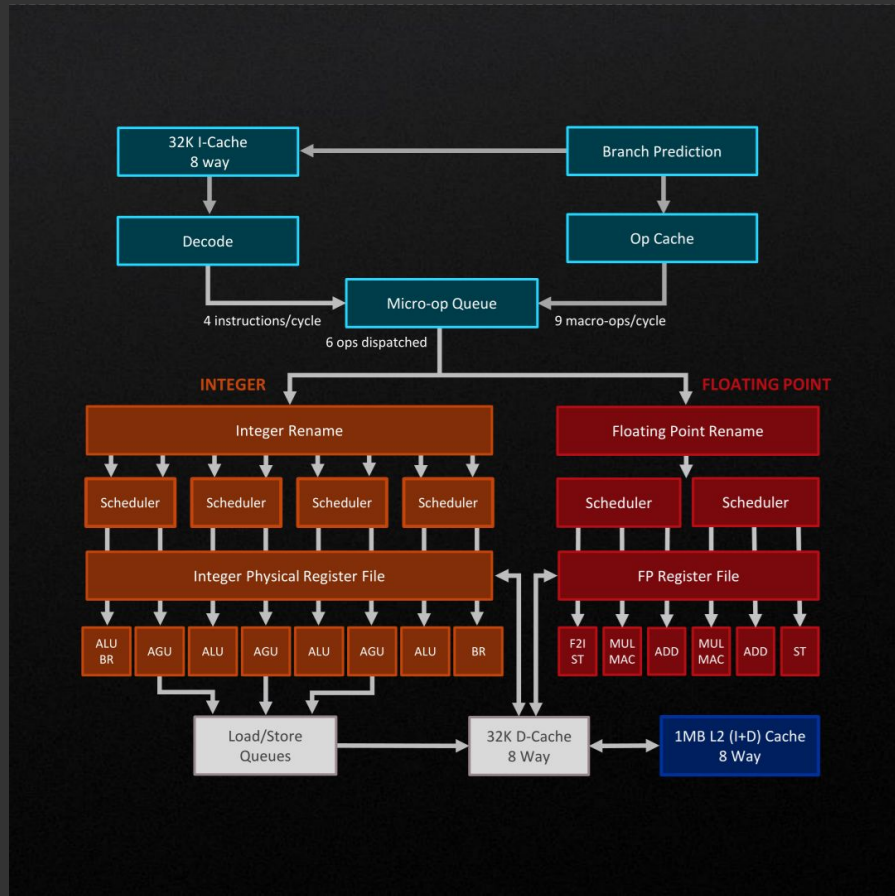
Performance

1. Number of instructions
 - a. Algorithmic complexity
 - b. Waste
2. Speed at which instructions go through the CPU
 - a. Data locality
 - b. Multi-threading
 - c. SIMD



How does SIMD make my code faster?

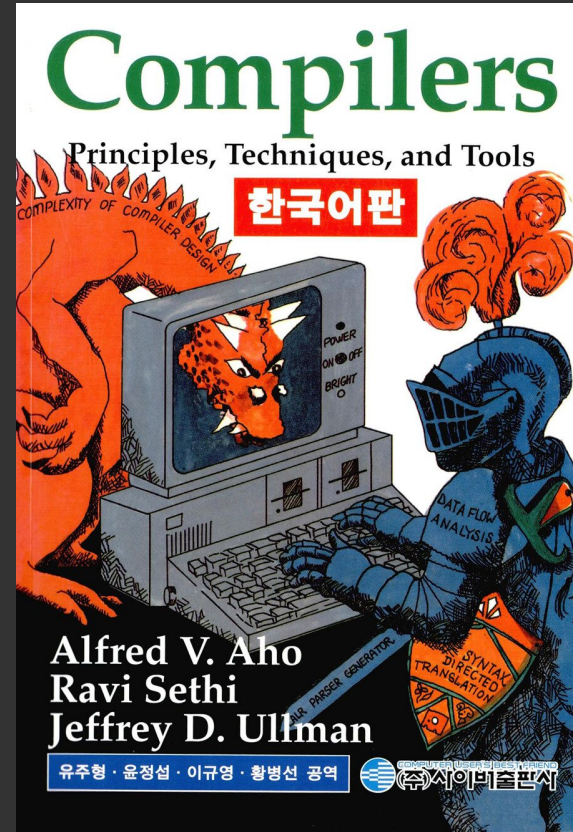
- Faster execution - do more useful work at the same time by utilizing vector hardware.
- Reduced cost of decoding - only one instruction to decode instead of multiple.
- Maximizes cache bandwidth - fewer loads per operation.



Auto-vectorization

If certain criteria are met, a compiler may be able to vectorize your code.

Free optimization? Yes, but compilers usually need some help to be able to auto-vectorize.



Auto-vectorization

1. **Loop vectorizer** unrolls loops and writes them as SIMD
2. **Block vectorizer** (a.k.a. **SLP**) merges multiple scalars into a vector in a block of code

<https://www.intel.com/content/dam/development/external/us/en/documents/31848-compilerautovectorizationguide.pdf>

<https://llvm.org/docs/Vectorizers.html>

Normal loop:

```
for (int i = 0; i < 1024; i++)  
{  
    a[i] = i;  
}
```

Unrolled loop:

```
for (int i = 0; i < 256; i += 4)  
{  
    a[i] = i;  
    a[i + 1] = i + 1;  
    a[i + 2] = i + 2;  
    a[i + 3] = i + 3;  
}
```



Back to demo

History

SSE (1999): 128-bit, 8 registers

SSE2 (2000): doubles and integers

AVX (2011): 256-bit

AVX2 (2013): integer instructions

AVX512 (2016): 512-bit

AVX-512 register scheme as extension
from the AVX (YMM0-YMM15) and SSE
(XMM0-XMM15) registers

511	256	255	128	127	0
ZMM0		YMM0		XMM0	
ZMM1		YMM1		XMM1	
ZMM2		YMM2		XMM2	
ZMM3		YMM3		XMM3	
ZMM4		YMM4		XMM4	
ZMM5		YMM5		XMM5	
ZMM6		YMM6		XMM6	
ZMM7		YMM7		XMM7	
ZMM8		YMM8		XMM8	
ZMM9		YMM9		XMM9	
ZMM10		YMM10		XMM10	
ZMM11		YMM11		XMM11	
ZMM12		YMM12		XMM12	
ZMM13		YMM13		XMM13	
ZMM14		YMM14		XMM14	
ZMM15		YMM15		XMM15	

Availability

Every 64-bit x86 processor in the world supports at least SSE and SSE2.

Call `__cpuid` to check which instruction sets are available (at runtime).

SSE2	100.00%	-16.30%
SSE3	100.00%	-16.49%
CMPXCHG16B	100.00%	-18.58%
LAHF / SAHF	100.00%	-18.78%
SSSE3	99.87%	-16.67%
SSE4.1	99.82%	-16.85%
SSE4.2	99.75%	-17.01%
AES	98.75%	-18.00%
FCMOV	98.60%	-15.71%
AVX	97.48%	-16.57%
NTFS	96.45%	-17.92%
AVX2	94.67%	-16.15%
HyperThreading	88.91%	-16.73%
SHA	71.14%	-11.73%
SSE4a	36.36%	-7.31%
AVX512CD	14.87%	-2.47%
AVX512F	14.87%	-2.44%
AVX512VNNI	14.84%	-2.47%
PrefetchW	0.00%	0.00%
AVX512ER	0.00%	0.00%
AVX512PF	0.00%	0.00%

SIMD in C++

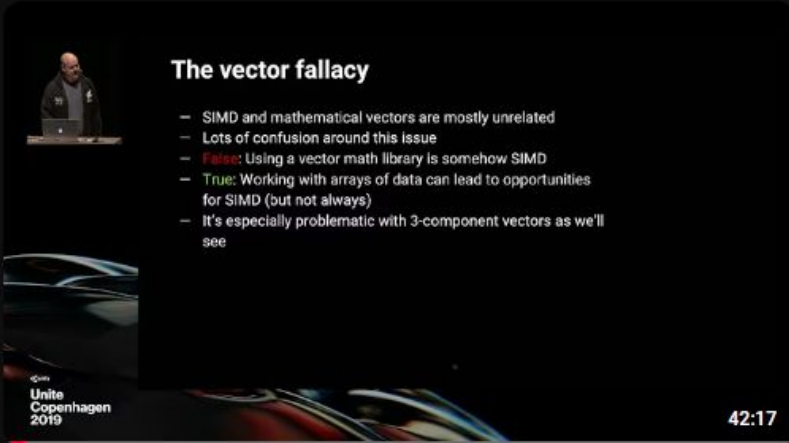
A SIMD library is coming in C++26.

For now:

- Intrinsics
- SIMD Libraries (Google Highway, xsimd, SIMD Everywhere)
- SIMD Math Libraries (DirectXMath, Eigen, glm)

SIMD in Unity

- Unity.Mathematics in Burst-compiled context
- Intrinsics in Burst-compiled context
- Native plugins
- Vector from System.Numerics when Unity transitions from Mono to .NET Core



The vector fallacy

- SIMD and mathematical vectors are mostly unrelated
- Lots of confusion around this issue
- **False:** Using a vector math library is somehow SIMD
- **True:** Working with arrays of data can lead to opportunities for SIMD (but not always)
- It's especially problematic with 3-component vectors as we'll see

Unite Copenhagen 2019

42:17

Intrinsics: Low-level engine development with Burst - Unite Copenhagen

10K views • 5 years ago

Unity

This session addresses how we are expanding the scope of the Burst Compiler to enable even...

Matching chapter 1:34 What is SIMD?

Intrinsics

Built-in functions, the implementation of which is handled by the compiler.

SIMD intrinsics often map to a single instruction, so it's kind of like writing inline assembly.

Include `<immintrin.h>` header!

```
float SumAVX2(std::span<float> input)
{
    __m256 sum = _mm256_setzero_ps();

    for (size_t i = 0; i < input.size(); i += 8)
    {
        sum = _mm256_add_ps(sum, _mm256_load_ps(&input[i]));
    }

    float result[8];
    _mm256_store_ps(result, sum);

    return result[0] + result[1] + result[2] + result[3] +
           result[4] + result[5] + result[6] + result[7];
}
```

Data Types

128-bit:

- `__m128`
- `__m128d`
- `__m128i`

`__m128i`



16-bit integer x 8

256-bit (AVX):

- `__m256`
- `__m256d`
- `__m256i`



32-bit integer x 4



64-bit integer x 2

Initialization

Load from memory or initialize with values.

- `_mm_load_ps`
- `_mm_loadu_ps`
- `_mm_setzero_ps`
- `_mm_set_ps`
- `_mm_set1_ps`

Init to zero

```
float SumAVX2(std::span<float> input)
{
    __m256 sum = _mm256_setzero_ps();

    for (size_t i = 0; i < input.size(); i += 8)
    {
        sum = _mm256_add_ps(sum, _mm256_load_ps(&input[i]));
    }

    float result[8];
    _mm256_store_ps(result, sum);

    return result[0] + result[1] + result[2] + result[3] +
           result[4] + result[5] + result[6] + result[7];
}
```

Load

Arithmetic

Quite self explanatory...

- `_mm_add_ps`
- `_mm_sub_ps`
- `_mm_mul_ps`
- `_mm_div_ps`

```
float SumAVX2(std::span<float> input)
{
    __m256 sum = _mm256_setzero_ps();

    for (size_t i = 0; i < input.size(); i += 8)
    {
        sum = _mm256_add_ps(sum, _mm256_load_ps(&input[i]));
    }

    float result[8];
    _mm256_store_ps(result, sum);

    return result[0] + result[1] + result[2] + result[3] +
           result[4] + result[5] + result[6] + result[7];
}
```

Add

Back to scalar

Store into an array or convert to a single scalar.

- `_mm_store_ps`
- `_mm_storeu_ps`
- `_mm_cvtss_f32`
- `_mm_extract_ps`

```
float SumAVX2(std::span<float> input)
{
    __m256 sum = _mm256_setzero_ps();

    for (size_t i = 0; i < input.size(); i += 8)
    {
        sum = _mm256_add_ps(sum, _mm256_load_ps(&input[i]));
    }

    Store
    _mm256_store_ps(result, sum);

    return result[0] + result[1] + result[2] + result[3] +
           result[4] + result[5] + result[6] + result[7];
}
```



Another Example

Learn More

Articles:

<http://const.me/articles/simd/simd.pdf>

<https://mcyoung.xyz/2023/11/27/simd-base64/>

Talks:

<https://gdcvault.com/play/1022248/SIMD-at-Insomniac-Games-How>

<https://www.youtube.com/watch?v=BpwvXkoFcp8>

Reference:

<https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html>

Glossary

Auto-vectorization SIMD

AVX SSE

Execution unit x86

Fetch-Decode-Execute

Instruction set architecture

Intrinsics

Microarchitecture

Register